

```
function optimize(data) {
```

```
// [FUENTE: TEMA 17 - ENTORNOS DE DESARROLLO]
```

```
}
```

```
function optimize(data) { ... }
```

```
import clean_code from 'refactor';
```

OPTIMIZACIÓN DE CÓDIGO

El arte de la refactorización para un software

limpio, rápido y escalable.

```
}
```

```
class ScalableSystem extends Core { ... }
```

```
var performance_metric = 0.98;
```

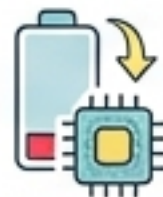
```
/* TODO: improve memory usage */
```

El Síndrome del Código Envejecido

Salud del Sistema

Lentitud Operativa

Consumo excesivo de memoria y batería en tiempo de ejecución.



Complejidad Paralizante

Código frágil donde añadir nuevas funcionalidades introduce errores en cascada.



Deuda Técnica

Falta de estándares de calidad que ahoga el mantenimiento.



Tiempo / Nuevas Funcionalidades

¿Qué es la Refactorización?

Cambio Interno (Lo que muta)

- Estructura y Organización
- Diseño Arquitectónico
- Consumo de Memoria (Rendimiento)

Contrato Externo (Lo que permanece)

- Comportamiento del sistema
- Funcionalidad para el usuario

Cambiamos la forma geométrica del código, nunca su función.

El Límite: ¿Refactorizar o Reescribir?

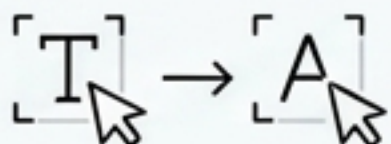
✓ Vía Libre (Cuándo Refactorizar)

- ✓ Para **reducir** la complejidad ciclomática.
- ✓ Para **mejorar** el **rendimiento** (ej. búsquedas en conjuntos).
- ✓ Para facilitar la **extensibilidad futura**.

⛔ Callejón sin Salida (Cuándo Reescribir)

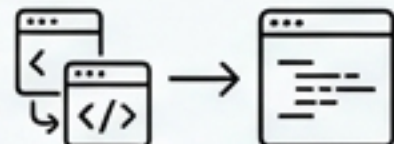
- ✗ El **diseño base** es desastroso e insalvable.
- ✗ Existe un **riesgo masivo** e **incontrolable** de romper dependencias ocultas.
- ✗ El **coste en tiempo** de la refactorización supera la creación de una solución nueva.

El Arsenal: 8 Patrones de Refactorización



Renombrado

Nombres representativos para clases y paquetes.



Sustitución de bloques

Conversión de bloques de código recurrentes en métodos reutilizables.



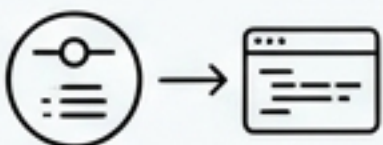
Campos encapsulados

Ocultación de asignaciones y consultas (Getters & Setters).



Movimiento de clases

Reubicación a nuevos paquetes con actualización automática.



Extracción de interfaz

Creación de contratos públicos para otras clases.



Cambio de parámetros

Modificación del número y tipo de acceso.



Mover a otro nivel

Ascenso de clases en la jerarquía (ej. hacerlas públicas).



Borrado seguro

Eliminación inteligente detectando dependencias previas.

Caso Práctico 1: La Trampa de los Nombres

Código Problemático

```
double aux = (altura + anchura)*2;  
Console.WriteLine(aux);  
Console.WriteLine(aux);  
  
temp = altura * anchura;
```

Código Limpio

```
double perimetro = (altura + anchura)*2;  
Console.WriteLine(perimetro);  
Console.WriteLine(perimetro);  
  
double area = altura * anchura;
```

 aux y temp son semillas de errores futuros. No aportan contexto semántico.

 Nombres con significado semántico absoluto que explican la intención.

 El código debe narrar su propio propósito.

La Red de Seguridad: Integración Continua

Detección (Analizadores)

- Revisión de estilo y métricas.
- Alertas de editor para reglas incumplidas.

Acción (Refactorización)

- Aplicación iterativa de patrones.
- Cambios pequeños y controlados.

Validación (Pruebas Unitarias)

- Testeo automático de funcionalidades.
- Garantía de comportamiento intacto.

Refactorizar sin cobertura de pruebas no es optimización, es jugar a la ruleta rusa con el código.

Automatización: Herramientas del IDE

Ingeniería Inversa

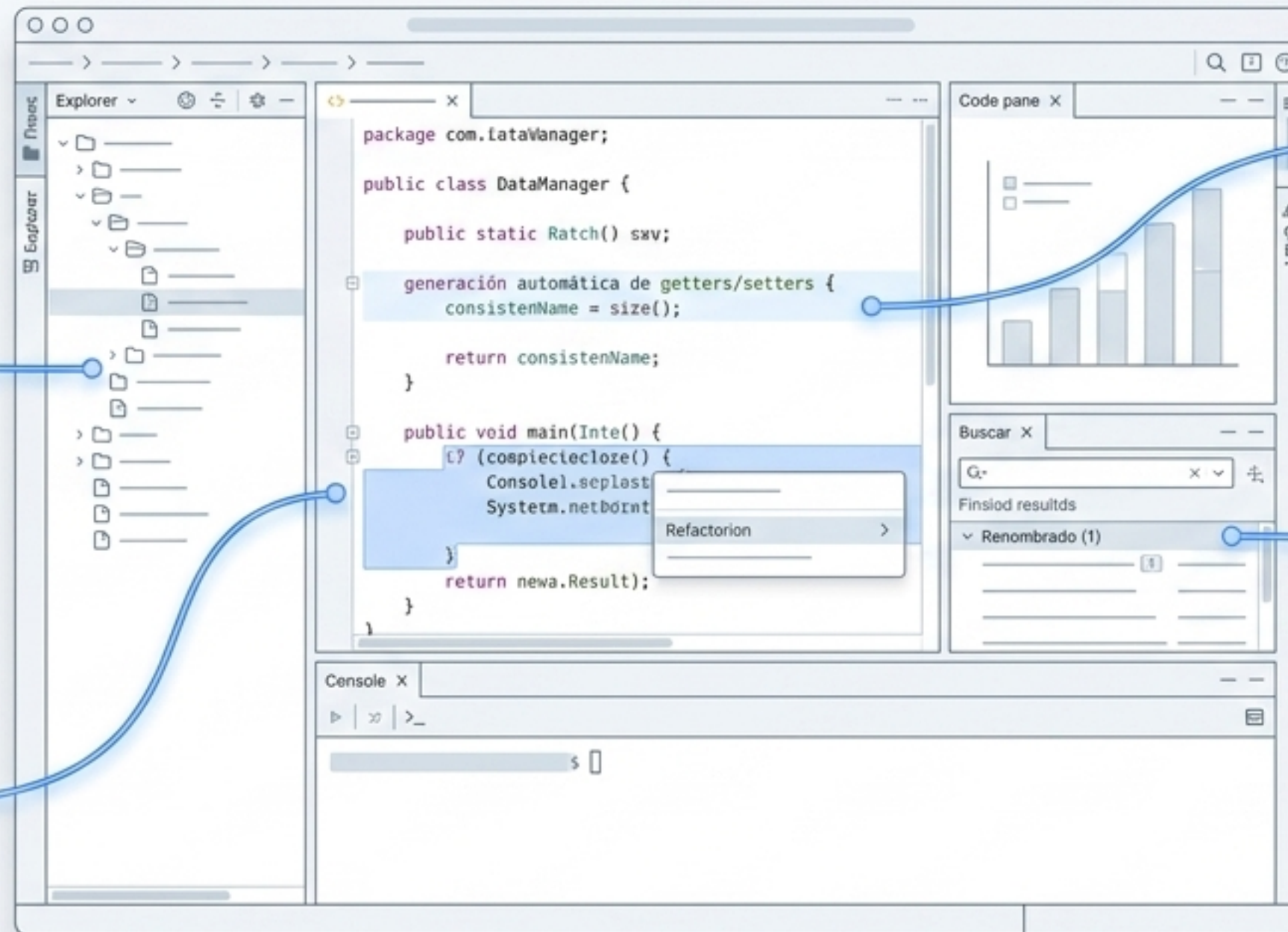
Representación visual de dependencias para comprender la arquitectura.

Abstracción Forzada

Generación automática de getters/setters y polimorfismo.

División Lógica

Extracción automática de fragmentos a nuevas clases.



Trazabilidad

Renombrado masivo y seguro en todo el proyecto.

Caso Práctico 2: Reestructurando la Jerarquía

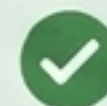
Clase: Coche



```
1 boolean concursante(Ejemplo e) {  
2     ...  
3 }
```

Migración Arquitectónica

Clase: Ejemplo (Gestor)



```
1 boolean concursantes(Coche x) {  
2     ...  
3 }
```

- **Violación de Responsabilidad Única:** Conocer la matriz entera de la carrera no es responsabilidad del vehículo.

- **Diseño Sólido:** Ahora la lógica reside en el gestor del evento, recibiendo el coche como parámetro.

Conclusión y Protocolo de Despliegue

- [✓] 1. **Diagnosticar:** Utilizar analizadores de código para auditar la deuda técnica.
- [✓] 2. **Asegurar:** Verificar la existencia de pruebas unitarias automatizadas.
- [✓] 3. **Ejecutar:** Aplicar patrones apoyándose estrictamente en las herramientas del IDE.
- [✓] 4. **Validar:** Confirmar que la funcionalidad externa es idéntica a la inicial.



La Regla de Oro en Producción

Antes de alterar código en producción, pide permiso explícito y define responsabilidades con la empresa asociada. Si algo se rompe tras la refactorización, la cadena de mando debe estar clara y notificada.